

Documento de Arquitectura de Sistemas

Motor de Optimización de Viajes Continuo y Soberano

Ingeniería de Sistemas - Diseño Full Stack

Agosto 2026

Resumen

Este documento detalla la arquitectura de un sistema de planificación y rastreo de viajes de alta frecuencia. El diseño prioriza la soberanía de los datos (100% de inferencia de IA local), la tolerancia a fallos mediante colas distribuidas y la resolución de lógica combinatoria utilizando microservicios en C++ residentes en memoria. Se aborda la transición de un sistema reactivo a uno proactivo (monitorización continua 24/7) integrando algoritmos de emparejamiento en memoria y análisis de series temporales.

Índice

1	Introducción, Visión del Producto y Objetivos	2
1.1	El Problema y la Visión	2
1.2	Casos de Uso Principales	2
1.3	Filosofía Arquitectónica	2
1.4	Características Clave del Sistema	2
2	Motor de Optimización Reactivo (C++ Core)	3
2.1	El Problema de Satisfacción de Restricciones (CSP)	3
2.2	Integración con Python	3
3	Orquestación Semántica y Gateway (Python/FastAPI)	3
3.1	Gateway de Conexiones	3
3.2	Enjambre Multi-Agente (LangGraph)	4
4	Subsistema de Búsqueda Continua (Distributed Cron)	4
4.1	Celery Beat y Workers Distribuidos	4
4.2	Estrategias de Evasión (Anti-Ban)	4
5	Motor de Emparejamiento en Memoria (In-Memory Matching)	4
5.1	Implementación del Interval Tree (C++)	4
6	Series Temporales y Detección de Anomalías	5
6.1	TimescaleDB	5
6.2	Detección Matemática de Anomalías (Z-Score)	5
7	Infraestructura y Conclusión	5

1 Introducción, Visión del Producto y Objetivos

1.1 El Problema y la Visión

La planificación de un viaje suele ser un proceso manual, tedioso y fragmentado. Los usuarios deben cruzar información de múltiples plataformas (vuelos, hoteles, actividades) para intentar encajar un rompecabezas logístico sujeto a restricciones estrictas de fechas y presupuestos.

El objetivo primordial de este proyecto es construir una plataforma integral y soberana que resuelva este problema mediante un enfoque dual: un **Motor Reactivo de Lenguaje Natural** y un **Motor Proactivo de Búsqueda Continua**. La aplicación está diseñada para actuar como un agente de viajes hiper-optimizado, capaz de razonar sobre preferencias complejas y cazar oportunidades en tiempo real sin intervención humana.

1.2 Casos de Uso Principales

La plataforma se sustenta sobre dos pilares funcionales de cara al usuario:

1. **Planificación Reactiva con Lenguaje Natural:** El usuario puede interactuar con la plataforma introduciendo prompts complejos y conversacionales, como por ejemplo: *"Quiero hacer un viaje a Roma del 1 al 5 de agosto, somos dos personas y tenemos un presupuesto máximo de 400 euros en total. Nos gusta el arte moderno"*. El sistema procesará esta petición, extraerá las restricciones y buscará la mejor combinación matemática (vuelos + hotel + actividades) que maximice la calidad del viaje garantizando que no se exceda el límite presupuestario.
2. **Monitorización Proactiva en Segundo Plano (Stream Processing):** El usuario configura zonas de salida preferidas y rangos de precios generales (ej. *"Avísame de cualquier combinación de vuelo y hotel en capitales europeas por menos de 150 euros para un fin de semana"*). El sistema funcionará 24/7 de forma invisible, ingiriendo decenas de miles de precios de proveedores, combinándolos y cruzándolos constantemente con las alertas del usuario. Al detectar una anomalía estadística (una verdadera "ganga"), enviará una notificación inmediata con el paquete completo ya armado.

1.3 Filosofía Arquitectónica

Para lograr estos objetivos a nivel empresarial y garantizar un sistema robusto, el desarrollo evita el error común de delegar la lógica dura a los Modelos de Lenguaje Grande (LLMs). Los modelos generativos son propensos a alucinaciones matemáticas y errores de cálculo en problemas combinatorios.

La arquitectura se basa en la **Soberanía de Datos y Computación**: la IA se ejecuta localmente y se relega estrictamente al procesamiento de lenguaje natural y extracción de entidades. Todo el razonamiento matemático, el cálculo de presupuestos y el emparejamiento de ofertas en tiempo real recae sobre microservicios deterministas en C++ de ultra-baja latencia.

1.4 Características Clave del Sistema

- **Cero Dependencias de IA Externa:** Uso exclusivo de modelos open-source cuantizados (ej. Llama 3.1 o Qwen 2.5) ejecutados en el hardware local.
- **Análisis Combinatorio Exacto:** Búsqueda de la "mejor opción" real mediante algoritmos estables de Optimización de Restricciones en C++, no mediante estimaciones generativas.

- **Procesamiento Continuo (24/7):** Ingesta ininterrumpida de fluctuaciones de precios en el mercado turístico, tolerante a fallos y bloqueos.
- **Evaluación $O(\log N)$ en Memoria:** Las miles de alertas de los usuarios se resuelven en memoria RAM mediante árboles de intervalos, logrando un rendimiento equivalente a sistemas de *High-Frequency Trading*.

2 Motor de Optimización Reactivo (C++ Core)

Cuando un usuario solicita la creación de un itinerario en un momento específico, el sistema debe encajar vuelos, hoteles y puntos de interés.

2.1 El Problema de Satisfacción de Restricciones (CSP)

La construcción de un itinerario se modela como una variante del *Time-Constrained Traveling Salesperson Problem (TSPTW)* combinado con el Problema de la Mochila (Knapsack) para el presupuesto.

Se implementa en C++20 utilizando Programación Dinámica con Máscaras de Bits (*Bitmasking*). El estado se representa típicamente por una máscara que indica qué puntos de interés han sido visitados, el nodo actual, y el tiempo transcurrido.

La función de coste multi-objetivo a minimizar es:

$$f(x) = \alpha \cdot C(x) + \beta \cdot T(x) - \gamma \cdot S(x) \quad (1)$$

Donde:

- $C(x)$: Coste total económico (vuelos + hoteles + entradas).
- $T(x)$: Tiempo total desperdiciado en tránsito y escalas.
- $S(x)$: Puntuación o *score* de calidad de los lugares visitados.
- α, β, γ : Pesos dinámicos configurados por el usuario.

2.2 Integración con Python

Para evitar la sobrecarga de serialización HTTP interna, el motor C++ se compila como una librería compartida dinámica (.so) utilizando `pybind11`. Esto permite a FastAPI invocar la función de cálculo como una función nativa de Python, logrando tiempos de ejecución inferiores a 50 milisegundos para grafos de hasta 25 nodos (suficiente para un viaje de 5-7 días).

3 Orquestación Semántica y Gateway (Python/FastAPI)

3.1 Gateway de Conexiones

FastAPI expone conexiones por *WebSockets*. Esto es crítico: las peticiones pueden tardar varios segundos (debido al scraping y la inferencia local). El WebSocket mantiene una vía abierta para transmitir eventos granulares al cliente: ["INICIANDO_INFERENCIA", "SCRAPEANDO_RYANAIR", "EVALUANDO_RUTAS"].

3.2 Enjambre Multi-Agente (LangGraph)

Se utiliza `vLLM` u `Ollama` para exponer una API local en el puerto 11434. El grafo de LangGraph orquesta a los agentes:

- **Router Agent:** Analiza el prompt y determina si es una petición reactiva ("Crea un viaje") o proactiva ("Avísame si el vuelo baja de precio").
- **RAG Agent (Knowledge Retrieval):** Consulta la base de datos vectorial (PostgreSQL con `pgvector`) para obtener reglas locales (ej. "Los museos en París cierran los martes").
- **Validator Agent (El Guardián):** Valida la salida del modelo contra un esquema estricto de Pydantic. Si el modelo alucina (ej. asigna 400 dólares a un presupuesto que el usuario dijo que era de 200 euros), el Validator captura el error y re-promptea al modelo local automáticamente.

4 Subsistema de Búsqueda Continua (Distributed Cron)

Para evolucionar a un sistema 24/7 de alta escala empresarial, se implementa un clúster de ingesta continua.

4.1 Celery Beat y Workers Distribuidos

`Celery Beat` orquesta el cronograma. Cada N minutos, dispara tareas a una cola de Redis. Los *Celery Workers* recogen estas tareas y ejecutan scripts de scraping (usando bibliotecas como `HTTPX` o `Playwright` para renderizado JS).

4.2 Estrategias de Evasión (Anti-Ban)

El mayor desafío del scraping continuo es el bloqueo de IP. La arquitectura incluye:

- **Rotación de Proxies:** Uso de pools de proxies residenciales enrutados aleatoriamente por cada petición.
- **Exponential Backoff & Jitter:** Si un dominio devuelve un error HTTP 429 (Too Many Requests), el worker suspende ese dominio durante $2^c + \text{ruido}$ segundos antes de reintentar, evitando ráfagas predecibles.

5 Motor de Emparejamiento en Memoria (In-Memory Matching)

Cuando se obtienen 1000 nuevos precios de vuelos, verificar en una base de datos SQL (ej. PostgreSQL) si alguno de esos precios satisface alguna de las 50,000 alertas de usuarios configuradas destruiría el rendimiento del disco (I/O Bottleneck).

5.1 Implementación del Interval Tree (C++)

Este subsistema reside eternamente en la memoria RAM. Las alertas de los usuarios (ej. "Vuelo a Londres entre el 10 y el 20 de Agosto, máximo 50 euros") se cargan en un **Segment Tree 2D** o **Interval Tree**.

Cuando el worker de Celery obtiene un nuevo dato de vuelo (Destino=LHR, Fecha=15-Aug, Precio=35):

1. Se inyecta el dato al demonio C++ por gRPC o ZeroMQ.

2. El árbol binario busca intervalos superpuestos.
3. Encuentra a los usuarios afectados en una complejidad de tiempo de $O(\log N + K)$, donde N es el número de alertas y K el número de matches reales.
4. Emite los IDs de usuario de vuelta a Celery para que se envíen los correos/push notifications.

6 Series Temporales y Detección de Anomalías

No todos los vuelos por debajo del presupuesto del usuario son "buenas ofertas". Para garantizar la calidad, el sistema implementa análisis histórico.

6.1 TimescaleDB

Se utiliza la extensión TimescaleDB sobre PostgreSQL, creando *hyper-tables* particionadas por tiempo. Esto permite ingestar miles de precios por minuto eficientemente y ejecutar consultas analíticas sobre ventanas temporales.

6.2 Detección Matemática de Anomalías (Z-Score)

Antes de emitir una alerta "Gran Oferta", el sistema extrae el precio actual (X) y consulta en TimescaleDB la media (μ) y la desviación estándar (σ) del mismo vuelo en los últimos 30 días. Se calcula la puntuación Z (Z-Score):

$$Z = \frac{X - \mu}{\sigma} \quad (2)$$

Si $Z \leq -2.0$ (el precio actual está dos desviaciones estándar por debajo de la media histórica), el sistema clasifica algorítmicamente el hallazgo como una anomalía de alta confianza, ignorando falsos positivos.

7 Infraestructura y Conclusión

El ecosistema completo se despliega utilizando `docker-compose` en un servidor host (mínimo 16GB RAM/VRAM para acomodar el LLM, Redis, PostgreSQL y motores C++). Las redes de contenedores están aisladas; únicamente el puerto del Gateway (Nginx/FastAPI) se expone a internet. El acceso administrativo se orquesta a través de una VPN de malla (Tailscale), cerrando todos los puertos físicos mediante el cortafuegos `ufw`.

Este diseño representa la cúspide de la ingeniería de software moderna: combina resiliencia de sistemas distribuidos, optimización de hardware a nivel de sistema (C++) y orquestación inteligente (IA soberana), transformando un simple planificador en un motor financiero de oportunidades de viaje.